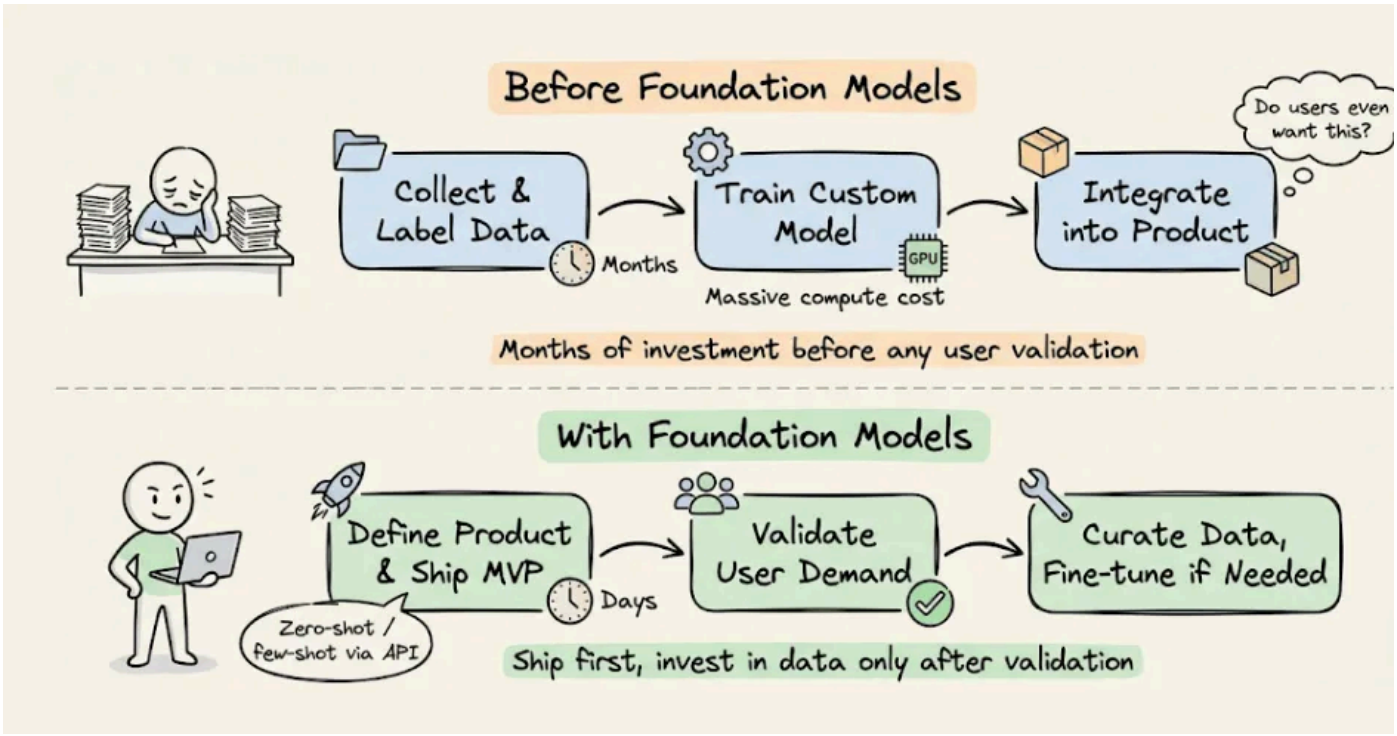
The Anatomy of an Agent Harness

A deep dive into what Anthropic, OpenAI, Perplexity and LangChain are actually building.

AVI CHAWLA
APR 07, 2026

The Canvas Framework: A structured approach to building AI agents that reach production

Before foundation models, building an AI feature involved collecting and labeli training data, training a custom model from scratch, and only then integrating into a product. This took months and a massive compute investment before tea could even test whether users wanted the feature.



Foundation models removed that bottleneck because they come pre-trained an accessible via API. Teams can now call GPT-4 or Claude with zero-shot or few-shot prompts, ship an MVP in days, validate user demand first, and only then invest in curating data for RAG or fine-tuning.

But for agentic systems, there's a missing layer.

Agent design needs to come right after defining the product, because the agent capabilities, workflows, and memory requirements are what determine what knowledge it needs and which model providers make sense downstream.

MongoDB published a detailed breakdown of the **Canvas Framework** built arou this exact sequence. It uses two planning canvases.

- The POC canvas has 8 squares covering product validation, agent design (capabilities, autonomy boundaries, memory requirements), data requireme (knowledge sources, update frequency, feedback loops), and model integrat (provider selection, prompt strategy, cost validation)
- The production canvas adds 11 squares for scaling, including fault toleranc multi-agent coordination, unified data architecture across application stora vector search, and agent memory, plus security hardening and governance.

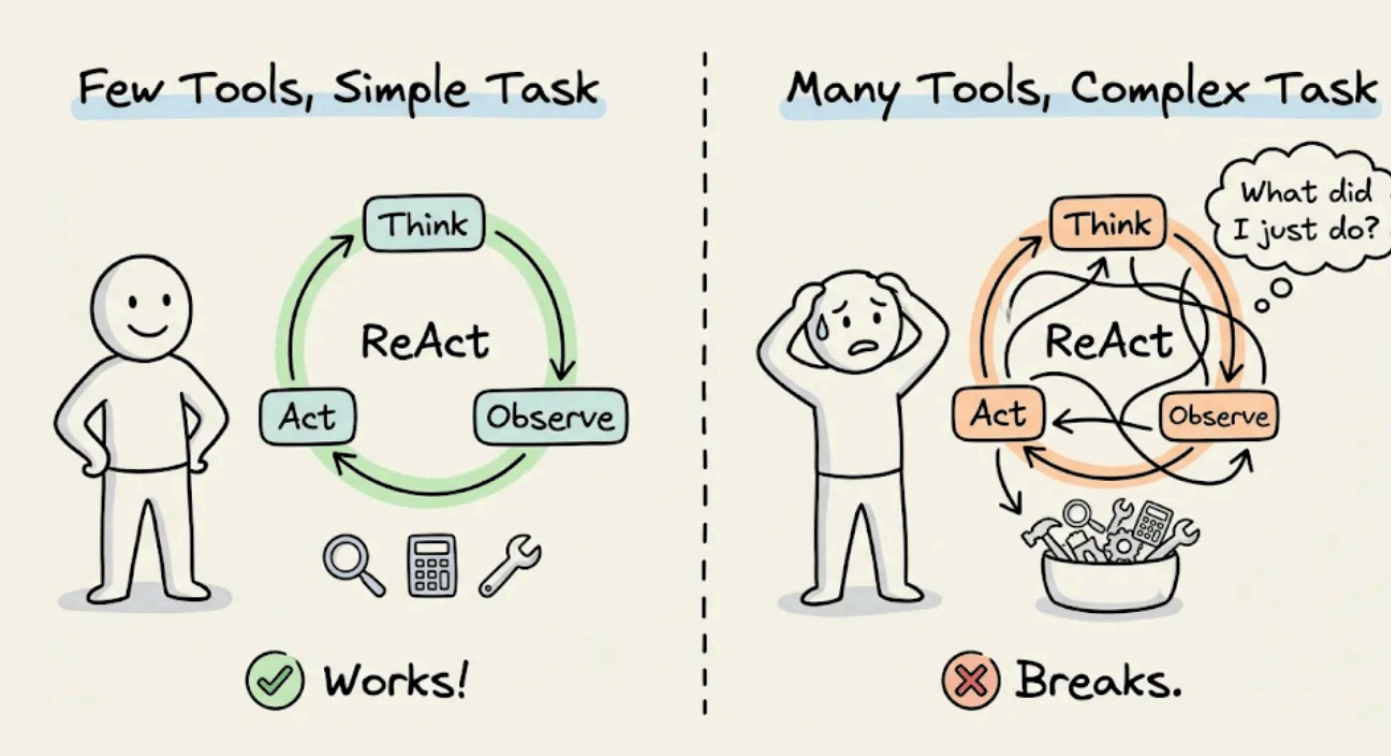
You can read the full breakdown here →

Thanks to MongoDB for partnering today!

The Anatomy of an Agent Harness

A **ReAct loop**, a couple of tools, and a well-written system prompt can get surprisingly far in a demo.

But the moment the task requires 10+ steps, things fall apart like the model forgets what it did three steps ago, tool calls fail silently, and the context windo fills up with garbage.



The problem isn't the model. It's everything around the model.

LangChain proved this when they changed only the infrastructure wrapping th LLM (same model, same weights) and jumped from outside the top 30 to rank 5 TerminalBench 2.0.

A separate research project hit a 76.4% pass rate by having an LLM optimize th infrastructure itself, surpassing hand-designed systems.

That infrastructure has a name now: the agent harness.

What is Agent Harness?

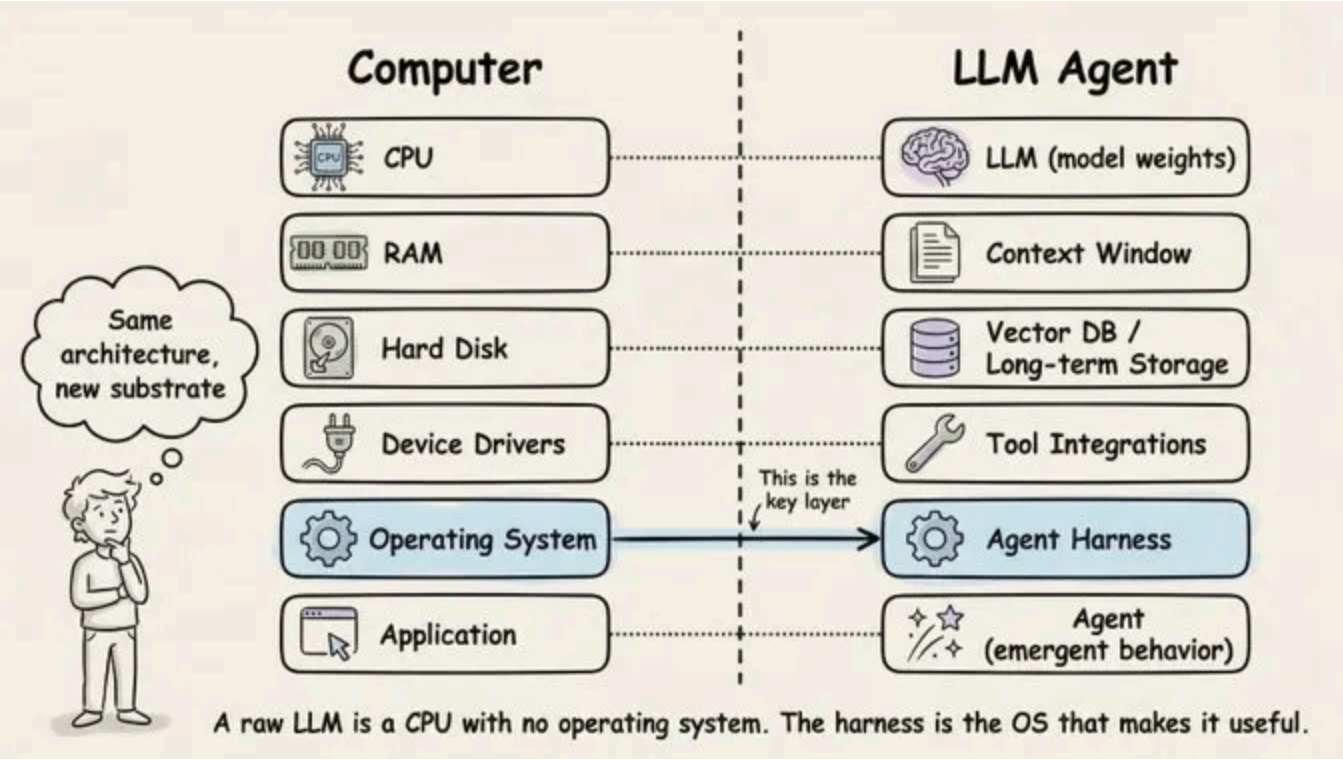
The term was formalized in early 2026, but the concept existed long before.

The harness is the complete software infrastructure wrapping an LLM, includi the orchestration loop, tools, memory, context management, state persistence, error handling, and guardrails.

Anthropic's Claude Code documentation puts it simply: the SDK is "the agent harness that powers Claude Code."

We really liked the canonical formula, from LangChain's Vivek Trivedy: "If you not the model, you're the harness."

To put it another way, the "agent" is the emergent behavior: the goal-directed, tool-using, self-correcting entity the user interacts with. The harness is the machinery producing that behavior. When someone says "I built an agent," they mean they built a harness and pointed it at a model.



Beren Millidge made this analogy precise in his 2023 essay:

- A raw LLM is a CPU with no RAM, no disk, and no I/O.
- The context window serves as RAM (fast but limited).
- External databases function as disk storage (large but slow).

- Tool integrations act as device drivers.

The harness is the operating system.

Three levels of engineering

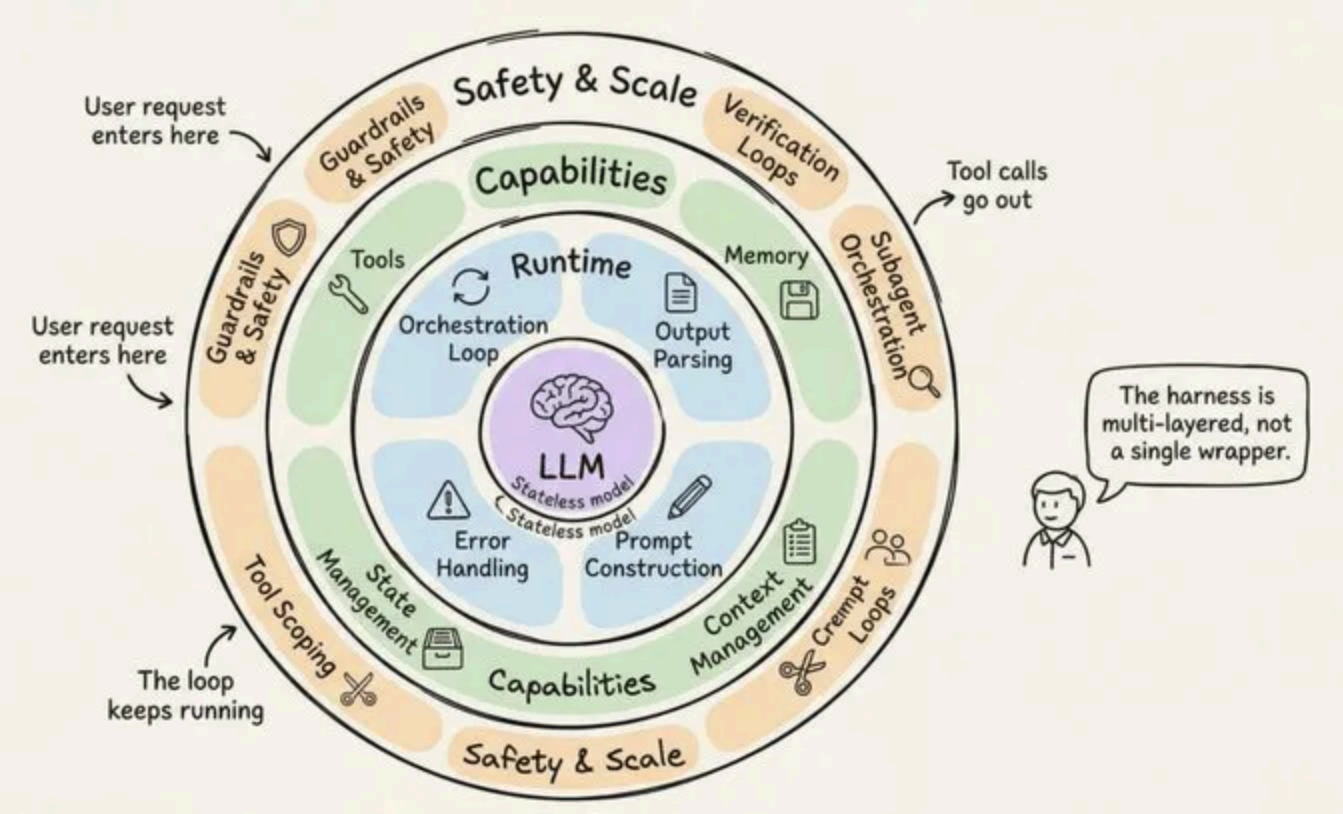
Three concentric levels of engineering surround the model:

- Prompt engineering crafts the instructions the model receives.
- Context engineering manages what the model sees and when.
- Harness engineering encompasses both, plus the entire application infrastructure: tool orchestration, state persistence, error recovery, verification loops, safety enforcement, and lifecycle management.

The harness is not a wrapper around a prompt. It is the complete system that makes autonomous agent behavior possible.

The 11 components of a production Harness

Synthesizing across Anthropic, OpenAI, LangChain, and the broader practitioner community, a production agent harness has eleven distinct components. Let's walk through each one.



1. The Orchestration Loop

This is the heartbeat. It implements the Thought-Action-Observation (TAO) cy also called the ReAct loop. The loop runs: assemble prompt, call LLM, parse output, execute any tool calls, feed results back, repeat until done.

Mechanically, it's often just a while loop. The complexity lives in everything the loop manages, not the loop itself. Anthropic describes their runtime as a "dumb loop" where all intelligence lives in the model. The harness just manages turns.

2. Tools

Tools are the agent's hands. They're defined as schemas (name, description, parameter types) injected into the LLM's context so the model knows what's available. The tool layer handles registration, schema validation, argument extraction, sandboxed execution, result capture, and formatting results back int LLM-readable observations.

Claude Code provides tools across six categories: file operations, search, execution, web access, code intelligence, and subagent spawning. OpenAI's Agents SDK supports function tools (via `function_tool`), hosted tools (WebSearch, CodeInterpreter, FileSearch), and MCP server tools.

3. Memory

Memory operates at multiple timescales. Short-term memory is the conversatio history within a single session. Long-term memory persists across sessions: Anthropic uses `CLAUDE.md` project files and auto-generated `MEMORY.md` files; LangGraph uses namespace-organized JSON Stores; OpenAI supports Session backed by SQLite or Redis.

Claude Code implements a three-tier hierarchy: a lightweight index (~150 characters per entry, always loaded), detailed topic files pulled in on demand, a raw transcripts accessed via search only.

4. Context management

This is where many agents fail silently. The core problem is context rot: model performance degrades 30%+ when key content falls in mid-window positions.

Even million-token windows suffer from instruction-following degradation as context grows.

Production strategies include:

- Compaction: summarizing conversation history when approaching limits (Claude Code preserves architectural decisions and unresolved bugs while discarding redundant tool outputs)
- Observation masking: JetBrains' Junie hides old tool outputs while keeping tool calls visible

- Just-in-time retrieval: maintaining lightweight identifiers and loading data dynamically (Claude Code uses grep, glob, head, tail rather than loading ful files)
- Sub-agent delegation: each subagent explores extensively but returns only 1,000 to 2,000 token condensed summaries

Anthropic's context engineering guide states the goal: find the smallest possibl set of high-signal tokens that maximize likelihood of the desired outcome.

5. Prompt construction

This assembles what the model actually sees at each step. It's hierarchical with system prompt, tool definitions, memory files, conversation history, and the current user message.

OpenAI's Codex uses a strict priority stack: server-controlled system message (highest priority), tool definitions, developer instructions, user instructions (cascading `AGENTS.md` files, 32 KiB limit), then conversation history.

6. Output parsing

Modern harnesses rely on native tool calling, where the model returns structure `tool_calls` objects rather than free-text that must be parsed.

The harness checks if there are any tool calls? If yes, it executes them and loops not, it gives the final answer.

For structured outputs, both OpenAI and LangChain support schema-constrai responses via Pydantic models.

Legacy approaches like RetryWithErrorOutputParser (which feeds the original prompt, the failed completion, and the parsing error back to the model) remain available for edge cases.

7. State management

LangGraph models state as typed dictionaries flowing through graph nodes, wi reducers merging updates.

Checkpointing happens at super-step boundaries, enabling resumption after interruptions and time-travel debugging.

OpenAI offers four mutually exclusive strategies: application memory, SDK sessions, server-side Conversations API, or lightweight previous_response_id chaining. Claude Code takes a different approach: git commits as checkpoints a progress files as structured scratchpads.

8. Error handling

Here's why this matters: a 10-step process with 99% per-step success still has o ~90.4% end-to-end success due to compounding.

LangGraph distinguishes four error types: transient (retry with backoff), LLM-recoverable (return error as ToolMessage so the model can adjust), user-fixable (interrupt for human input), and unexpected (bubble up for debugging). Anthro catches failures within tool handlers and returns them as error results to keep t loop running. Stripe's production harness caps retry attempts at two.

9. Guardrails and safety

OpenAI's SDK implements three levels: input guardrails (run on the first agent output guardrails (run on the final output), and tool guardrails (run on every too invocation).

A "tripwire" mechanism halts the agent immediately when triggered.

Anthropic separates permission enforcement from model reasoning architecturally. The model decides what to attempt; the tool system decides wh allowed. Claude Code gates ~40 discrete tool capabilities independently, with

three stages: trust establishment at project load, permission check before each tool call, and explicit user confirmation for high-risk operations.

10. Verification loops

This is what separates toy demos from production agents. Anthropic recomme three approaches: rules-based feedback (tests, linters, type checkers), visual feedback (screenshots via Playwright for UI tasks), and LLM-as-judge (a separa subagent evaluates output).

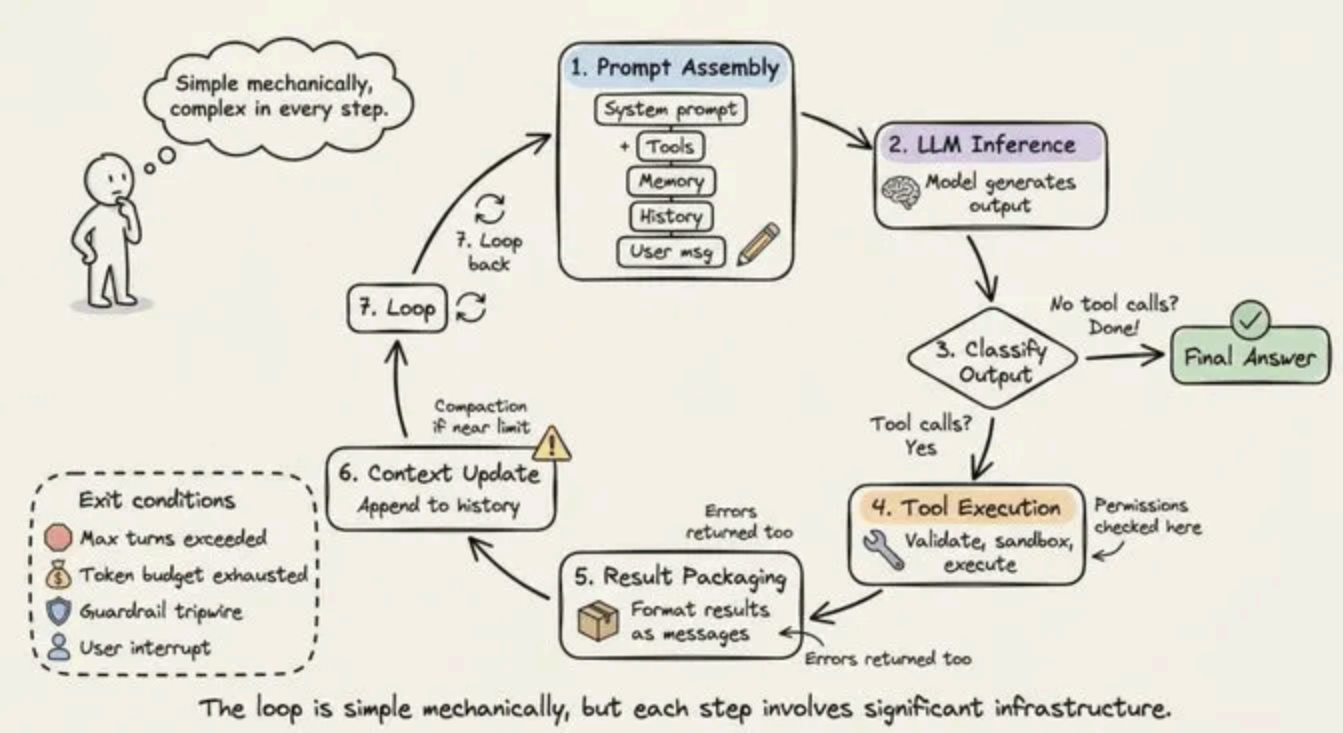
Boris Cherny, creator of Claude Code, noted that giving the model a way to ver its work improves quality by 2 to 3x.

11. Subagent orchestration

Claude Code supports three execution models: Fork (byte-identical copy of par context), Teammate (separate terminal pane with file-based mailbox communication), and Worktree (own git worktree, isolated branch per agent).

OpenAI's SDK supports agents-as-tools (specialist handles bounded subtask) a handoffs (specialist takes full control). LangGraph implements subagents as nested state graphs.

A step-by-step walkthrough



Now that you know the components, let's trace how they work together in a sin cycle.

- Step 1 (Prompt Assembly): The harness constructs the full input: system prompt + tool schemas + memory files + conversation history + current user message. Important context is positioned at the beginning and end of the prompt (the "Lost in the Middle" finding).
- Step 2 (LLM Inference): The assembled prompt goes to the model API. The model generates output tokens: text, tool call requests, or both.
- Step 3 (Output Classification): If the model produced text with no tool calls the loop ends. If it requested tool calls, proceed to execution. If a handoff w requested, update the current agent and restart.
- Step 4 (Tool Execution): For each tool call, the harness validates arguments, checks permissions, executes in a sandboxed environment, and captures results. Read-only operations can run concurrently; mutating operations ru serially.

- Step 5 (Result Packaging): Tool results are formatted as LLM-readable messages. Errors are caught and returned as error results so the model can self-correct.
- Step 6 (Context Update): Results are appended to the conversation history. I approaching the context window limit, the harness triggers compaction.
- Step 7 (Loop): Return to Step 1. Repeat until termination.

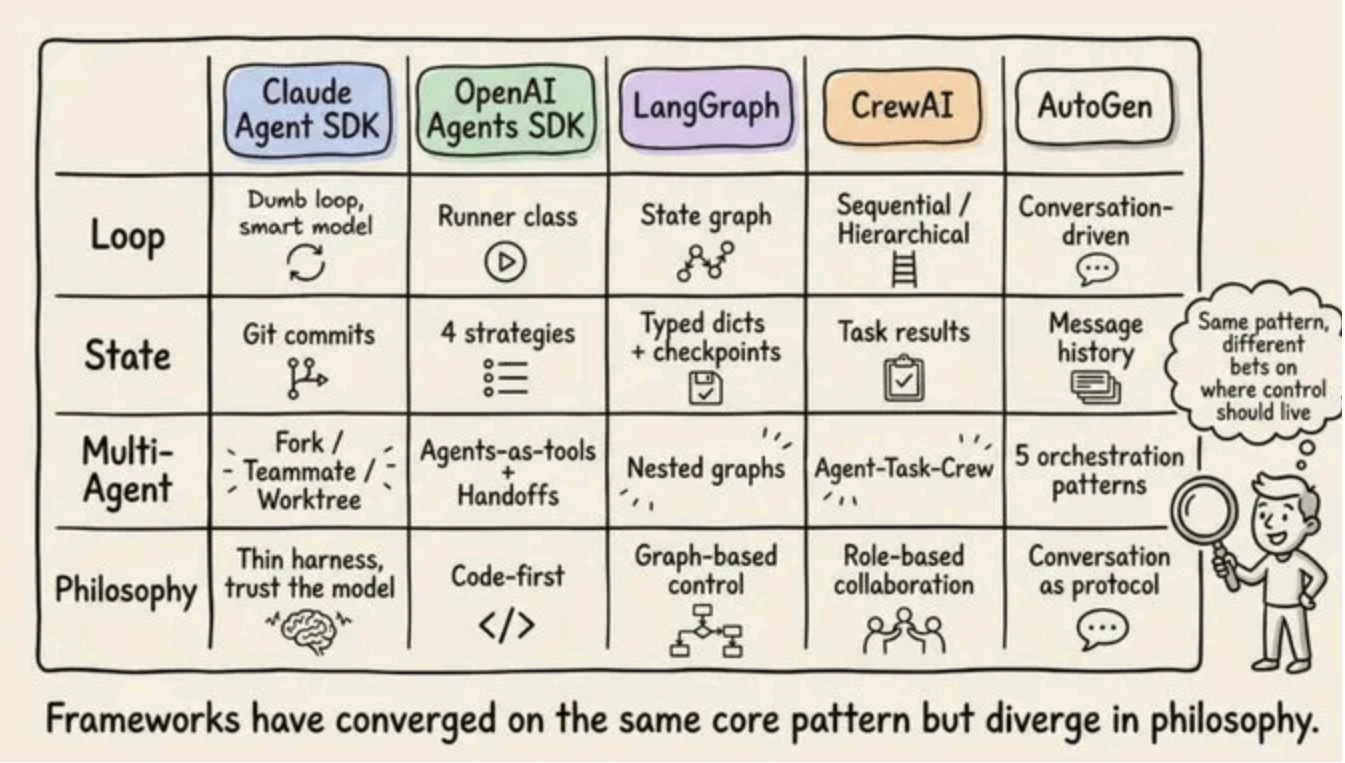
Termination conditions are layered: the model produces a response with no too calls, the maximum turn limit is exceeded, the token budget is exhausted, a guardrail tripwire fires, the user interrupts, or a safety refusal is returned. A simple question might take 1 to 2 turns. A complex refactoring task can chain dozens of tool calls across many turns.

For long-running tasks spanning multiple context windows, Anthropic develop a two-phase “Ralph Loop” pattern.

It uses an Initializer Agent that sets up the environment (init script, progress fi feature list, initial git commit), then a Coding Agent in every subsequent sessio reads git logs and progress files to orient itself, picks the highest-priority incomplete feature, works on it, commits, and writes summaries.

The filesystem provides continuity across context windows.

How frameworks implement the pattern



Anthropic's Claude Agent SDK exposes the harness through a `single query(` function that creates the agentic loop and returns an async iterator streaming messages.

The runtime is a "dumb loop." All intelligence lives in the model. Claude Code uses a Gather-Act-Verify cycle: gather context (search files, read code), take acti (edit files, run commands), verify results (run tests, check output), repeat.

OpenAI's Agents SDK implements the harness through the Runner class with three modes: async, sync, and streamed.

The SDK is "code-first": workflow logic is expressed in native Python rather th graph DSLs. The Codex harness extends this with a three-layer architecture: Codex Core (agent code + runtime), App Server (bidirectional JSON-RPC API), and client surfaces (CLI, VS Code, web app). All surfaces share the same harnes which is why "Codex models feel better on Codex surfaces than a generic chat window."

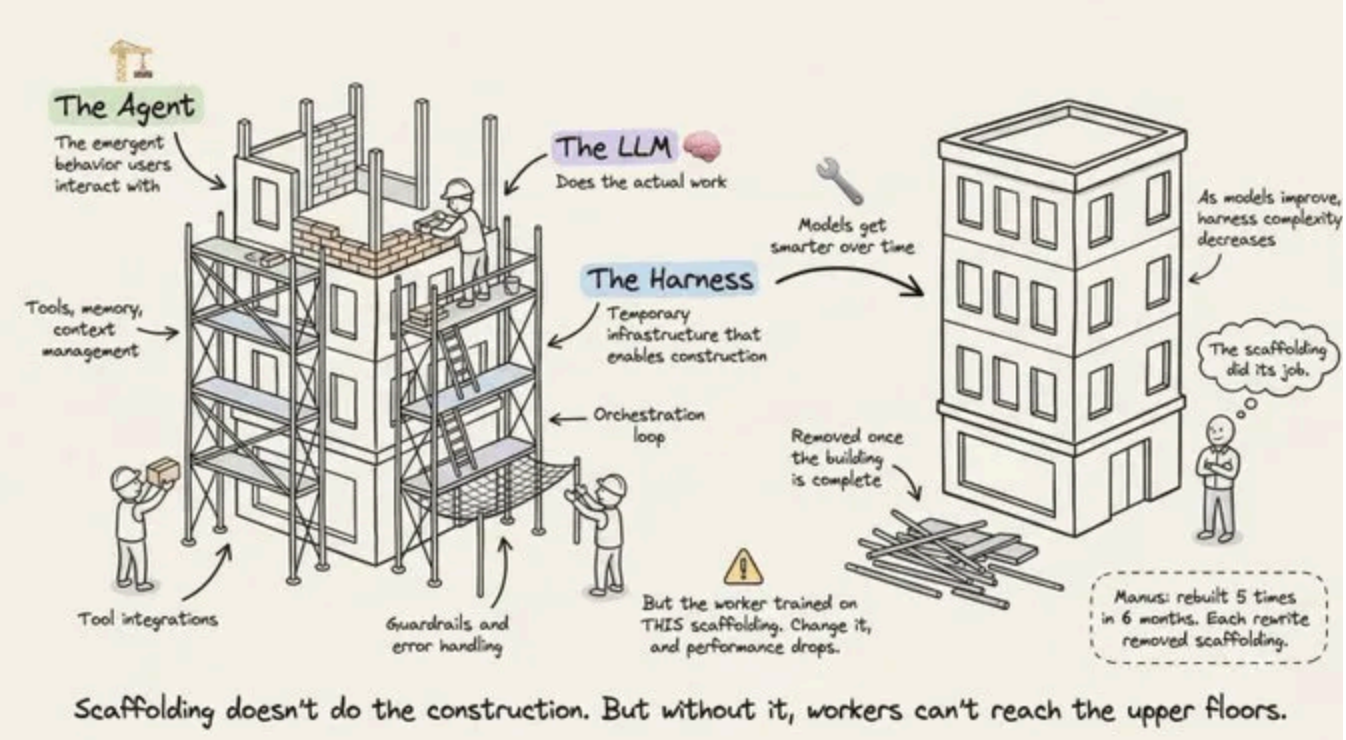
LangGraph models the harness as an explicit state graph. Two nodes (`llm_cal` and `tool_node`) connected by a conditional edge: if tool calls present, route to tool_node; if absent, route to END.

LangGraph evolved from LangChain's AgentExecutor, which was deprecated ir v0.2 because it was hard to extend and lacked multi-agent support. LangChain's Deep Agents explicitly use the term "agent harness": built-in tools, planning (write_todos tool), file systems for context management, subagent spawning, an persistent memory.

CrewAI implements a role-based multi-agent architecture: Agent (the harness around the LLM, defined by role, goal, backstory, and tools), Task (the unit of work), and Crew (the collection of agents). CrewAI's Flows layer adds a "deterministic backbone with intelligence where it matters," managing routing and validation while Crews handle autonomous collaboration.

The scaffolding metaphor

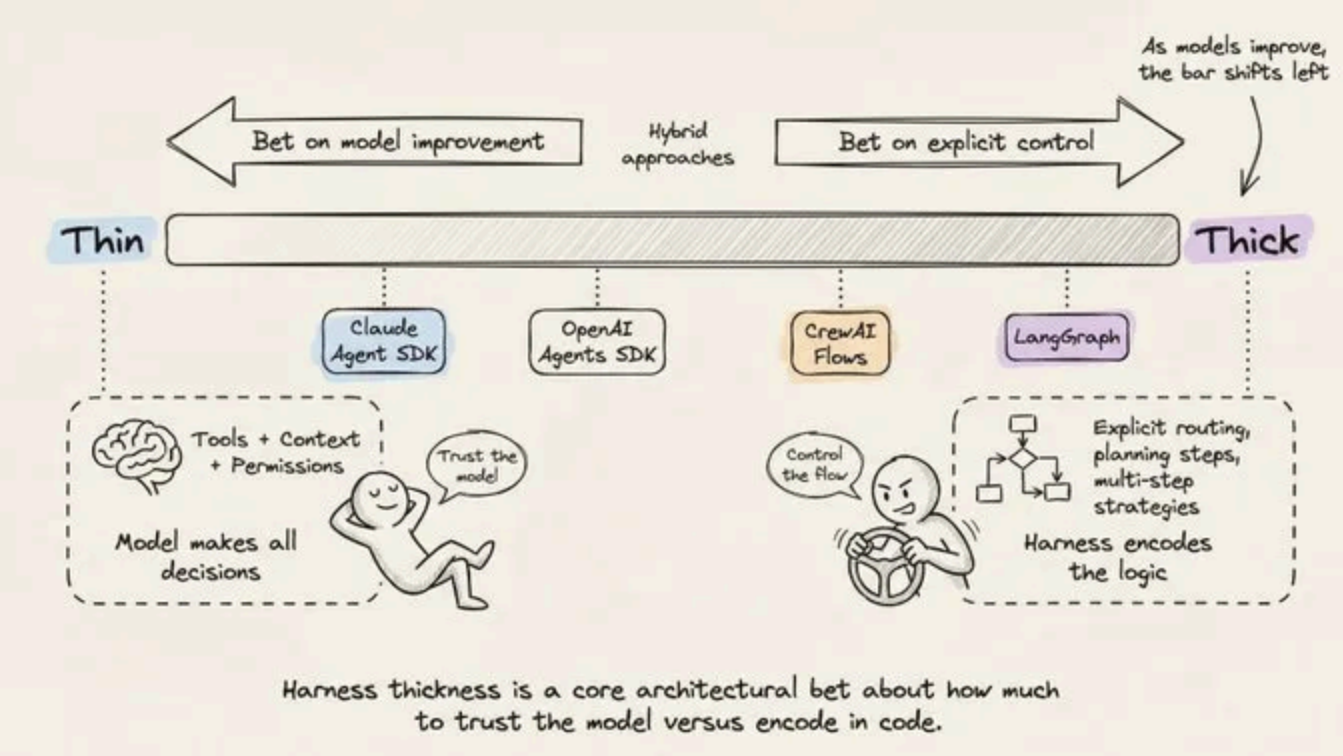
Construction scaffolding is a temporary infrastructure that enables workers to build a structure they couldn't reach otherwise. It doesn't do the construction. I without it, workers can't reach the upper floors.



The key insight is that scaffolding is removed when the building is complete. A models improve, harness complexity should decrease. Manus was rebuilt five times in six months, each rewrite removing complexity. Complex tool definitio became general shell execution. “Management agents” became simple structur handoffs.

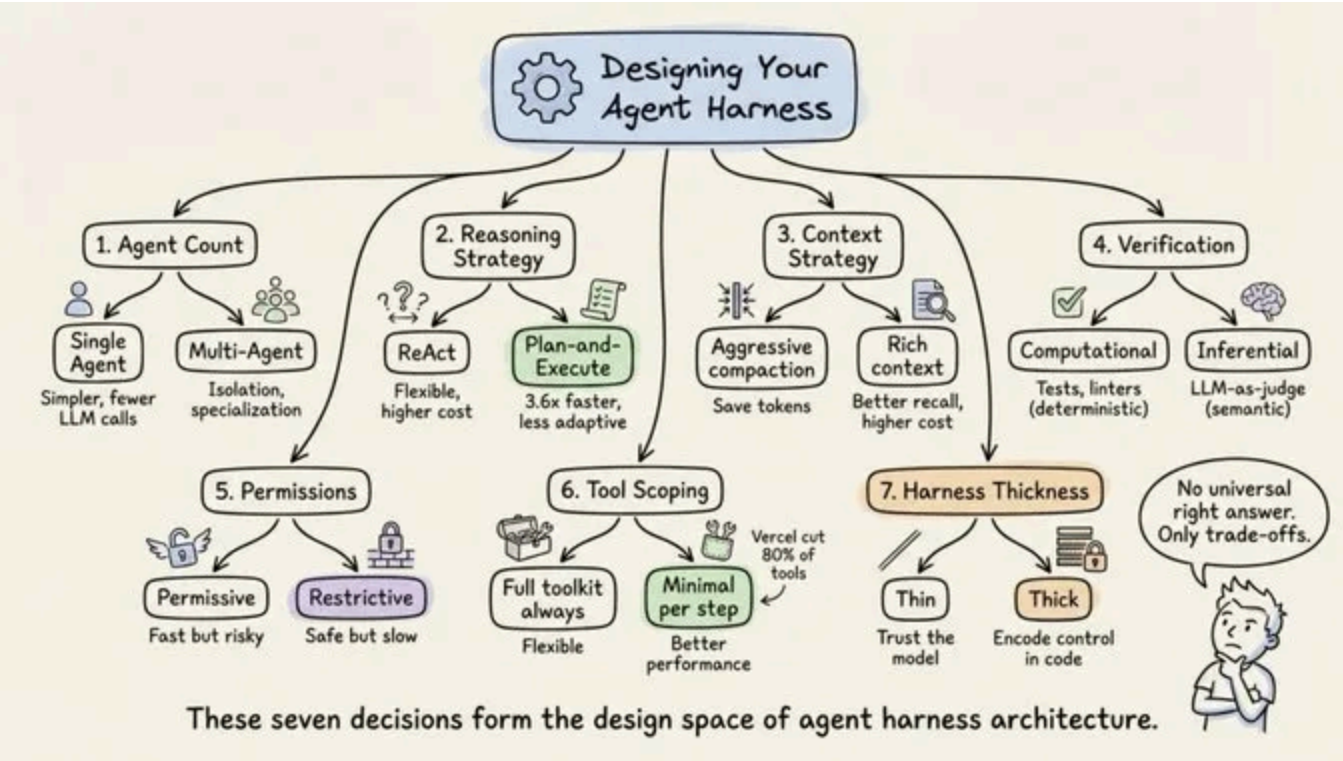
This points to the co-evolution principle where models are now post-trained w specific harnesses in the loop. Claude Code’s model learned to use the specific harness it was trained with. Changing tool implementations can degrade performance because of this tight coupling.

The future-proofing test for harness design states that if performance scales up with more powerful models without adding harness complexity, the design is sound.



Seven decisions for Harness definitions

Every harness architect faces seven choices:



1. Single-agent vs. multi-agent. Both Anthropic and OpenAI ask to maximize single agent first. Multi-agent systems add overhead (extra LLM calls for routing, context loss during handoffs). Split only when tool overload exceed ~10 overlapping tools or clearly separate task domains exist.
2. ReAct vs. plan-and-execute. ReAct interleaves reasoning and action at ever step (flexible but higher per-step cost). Plan-and-execute separates plannin from execution. LLMCompiler reports a 3.6x speedup over sequential ReAc
3. Context window management strategy. Five production approaches includ time-based clearing, conversation summarization, observation masking, structured note-taking, and sub-agent delegation. ACON research showed to 54% token reduction while preserving 95%+ accuracy by prioritizing reasoning traces over raw tool outputs.
4. Verification loop design. Computational verification (tests, linters) provide deterministic ground truth. Inferential verification (LLM-as-judge) catches semantic issues but adds latency. Martin Fowler's Thoughtworks team fran this as guides (feedforward, steer before action) versus sensors (feedback, observe after action).
5. Permission and safety architecture. Permissive (fast but risky, auto-approve most actions) versus restrictive (safe but slow, require approval for each action). The choice depends on the deployment context.
6. Tool scoping strategy. More tools often mean worse performance. Vercel removed 80% of tools from v0 and got better results. Claude Code achieves 95% context reduction via lazy loading. The principle: expose the minimum tool set needed for the current step.
7. Harness thickness. How much logic lives in the harness versus the model. Anthropic bets on thin harnesses and model improvement. Graph-based

frameworks bet on explicit control. Anthropic regularly deletes planning st from Claude Code's harness as new model versions internalize that capabil

The harness is the product

Two products using identical models can have wildly different performance bas solely on harness design. The TerminalBench evidence is clear that changing o the harness moved agents by 20+ ranking positions.

The harness is not a solved problem or a commodity layer. It's where the hard engineering lives like managing context as a scarce resource, designing verification loops that catch failures before they compound, building memory systems that provide continuity without hallucination, and making architectura bets about how much scaffolding to build versus how much to leave to the mod

The field is moving toward thinner harnesses as models improve. But the harne itself isn't going away. Even the most capable model needs something to manag its context window, execute its tool calls, persist its state, and verify its work.

The next time your agent fails, don't blame the model but rather look at the harness.

Thanks for reading!

18 Likes · 4 Restacks

← Previous

Next

Discussion about this post

Comments Restacks

Write a comment...

unlockz Apr 30

Should the arrows in the ReAct loop diagram represent 'Think ➡️ Act ➡️ Observe'?

LIKE REPLY